

Roteiro Prático - Aula 6

Git, PowerShell, Eclipse e camada Controller

Produto esperado

Repositório individual da Aula 6 criado pelo Classroom 50, contendo o código-base recebido pela sala online, histórico de commits, projeto Java Desktop funcionando no Eclipse, pacote controller criado, MenuAlunos usando AlunoController e README atualizado com responsabilidades e testes.

Regra principal da aula: cada estudante trabalha em seu próprio repositório. Faça uma alteração por vez, teste e somente depois registre o commit e envie ao GitHub.

Antes de começar

- Acesse o link do assignment da Aula 6 e aceite a atividade com a sua própria conta GitHub.
- Baixe a pasta com o código-base da Aula 6 disponibilizada na sala online da faculdade e extraia o arquivo ZIP antes de trabalhar.
- Confirme que o projeto contém .project, .classpath, README.md e a pasta src.
- Use o PowerShell para configurar e enviar o repositório; use o Eclipse para editar, compilar e executar o Java.
- Não crie um segundo repositório manualmente no GitHub. Use somente o repositório individual criado pelo Classroom 50.

Passo 1 - Aceitar o assignment e copiar a URL

1. Abra o link da atividade enviado pela professora.
2. Faça login na sua conta GitHub e aceite o assignment.
3. Aguarde a criação do seu repositório individual.
4. No repositório, clique em Code > HTTPS e copie a URL. Ela será usada mais adiante no PowerShell.

Passo 2 - Preparar a pasta do código-base

- Extraia o ZIP recebido pela sala online. Não trabalhe diretamente dentro do arquivo compactado.
- Abra a pasta principal do projeto e confirme se .project, .classpath, README.md e src estão nela.
- Essa será a pasta em que o Git será inicializado.

Passo 3 - Abrir o PowerShell e verificar o Git

No Explorador de Arquivos, abra a pasta do projeto. Clique na barra de endereço, digite powershell e pressione Enter.

```
git --version  
Get-ChildItem -Force
```

Atenção: se o comando git não for reconhecido, instale o Git for Windows antes de continuar. Se a listagem mostrar uma pasta .git, não execute git init novamente; chame a professora para conferir a origem dessa pasta.

Passo 4 - Criar o repositório Git local e fazer o primeiro envio

Execute os comandos abaixo na pasta do projeto. Substitua URL_DO_SEU_REPOSITORIO pela URL HTTPS copiada no Passo 1.

Comando	O que ele faz
<code>git init</code>	Inicializa o controle de versão na pasta local.
<code>git status</code>	Mostra quais arquivos ainda não estão sendo acompanhados.
<code>git add .</code>	Adiciona os arquivos do código-base ao próximo commit.
<code>git commit -m "adiciona codigo base da aula 6"</code>	Cria o primeiro registro do projeto.
<code>git branch -M main</code>	Garante que a branch principal se chame main.
<code>git remote add origin URL_DO_SEU_REPOSITORIO</code>	Conecta a pasta local ao repositório individual do Classroom 50.
<code>git remote -v</code>	Confere a URL configurada para envio e recebimento.
<code>git push -u origin main</code>	Envia o primeiro commit e vincula a branch local à remota.

Checkpoint 1 - GitHub

Atualize a página do seu repositório no navegador. Confirme se aparecem `src`, `README.md`, `.gitignore`, `.project` e `.classpath`, além do commit “adiciona codigo base da aula 6”.

Passo 5 - Importar o projeto no Eclipse

1. Abra o Eclipse no workspace da disciplina.
2. Acesse File > Import.
3. Escolha General > Existing Projects into Workspace e clique em Next.
4. Em Select root directory, selecione a mesma pasta em que você executou `git init`.
5. Deixe Copy projects into workspace desmarcado.
6. Confirme que o projeto é reconhecido e clique em Finish.

Passo 6 - Executar o código-base antes de alterar

- No Package Explorer, abra o projeto e localize `Principal.java`.
- Clique com o botão direito em `Principal.java` > Run As > Java Application.
- Teste cadastrar aluno, listar alunos, buscar aluno e sair.
- Se o projeto não executar, não altere o código. Chame a professora.

Passo 7 - Observar as camadas atuais

Pacote	Responsabilidade esperada
app	Iniciar o sistema e montar os objetos principais.
model	Representar os dados do domínio.
repository	Guardar e consultar os alunos em memória.
service	Concentrar regras simples e validações.
view	Conversar com o usuário usando JOptionPane.
controller	Receber pedidos da view e acionar o service. Será criado hoje.

Passo 8 - Criar o pacote controller e a classe AlunoController

- Clique com o botão direito sobre src > New > Package.
- Digite br.ceub.desenvolvimento.aula06.controller e clique em Finish.
- No novo pacote, escolha New > Class e crie AlunoController. Não marque public static void main.

```
package br.ceub.desenvolvimento.aula06.controller;

import br.ceub.desenvolvimento.aula06.service.AlunoService;

public class AlunoController {
    private AlunoService service;

    public AlunoController(AlunoService service) {
        this.service = service;
    }

    public String cadastrar(String nome, int semestre, String curso) {
        return service.cadastrar(nome, semestre, curso);
    }

    public String listarTodos() {
        return service.listarTodos();
    }

    public String buscarPorNome(String nome) {
        return service.buscarPorNome(nome);
    }

    public int quantidade() {
        return service.quantidade();
    }
}
```

Checkpoint 2 - Compilação

Salve a classe. Se houver linha vermelha, confira se todos os pacotes usam exatamente aula06 e se o import de AlunoService está correto.

Passo 9 - Ajustar Principal.java

```
package br.ceub.desenvolvimento.aula06.app;

import br.ceub.desenvolvimento.aula06.controller.AlunoController;
import br.ceub.desenvolvimento.aula06.repository.AlunoRepository;
import br.ceub.desenvolvimento.aula06.service.AlunoService;
import br.ceub.desenvolvimento.aula06.view.MenuAlunos;

public class Principal {
    public static void main(String[] args) {
        AlunoRepository repository = new AlunoRepository();
        AlunoService service = new AlunoService(repository);
        AlunoController controller = new AlunoController(service);
        MenuAlunos menu = new MenuAlunos(controller);
        menu.iniciar();
    }
}
```

Passo 10 - Ajustar MenuAlunos.java

Substitua o uso direto de AlunoService por AlunoController. Use Ctrl + F para localizar service e altere somente o que fizer sentido.

Local	Antes	Depois
import	import ...service.AlunoService;	import ...controller.AlunoController;
atributo	private AlunoService service;	private AlunoController controller;
construtor	MenuAlunos(AlunoService service)	MenuAlunos(AlunoController controller)
chamadas	service.listarTodos()	controller.listarTodos()

Passo 11 - Testar e registrar o primeiro commit da Aula 6

- Execute Principal.java.
- Cadastre um aluno válido.
- Liste os alunos.
- Busque o aluno cadastrado.
- Saia do sistema.

```
git status
git add src README.md
git commit -m "cria controller de alunos"
git push
```

Passo 12 - Atividade pontuada: opção 4

Agora você fará uma pequena evolução para demonstrar que compreendeu as responsabilidades entre view, controller e service.

- No menu textual, acrescente a opção 4 - Ver quantidade de alunos.
- Ajuste o limite de opções válidas de 0 a 4.
- Quando o usuário escolher 4, mostre a quantidade usando `controller.quantidade()`.
- Não consulte o repository diretamente pela view.
- Teste com zero alunos e depois com dois alunos cadastrados.

```
String menu = "1 - Cadastrar aluno\n"
            + "2 - Listar alunos\n"
            + "3 - Buscar aluno por nome\n"
            + "4 - Ver quantidade de alunos\n"
            + "0 - Sair";
opcao = lerInteiro(menu, 0, 4);

if (opcao == 1) {
    cadastrarAluno();
} else if (opcao == 2) {
    mostrar(controller.listarTodos());
} else if (opcao == 3) {
    String nome = lerTexto("Nome para busca:");
    mostrar(controller.buscarPorNome(nome));
} else if (opcao == 4) {
    mostrar("Total de alunos cadastrados: " + controller.quantidade());
}
```

Passo 13 - Atualizar README e enviar a entrega

- Preencha a tabela de responsabilidades no README.
- Registre os testes realizados e as dificuldades encontradas.
- Confira os arquivos alterados com `git status`.
- Faça o commit final e envie ao GitHub.

```
git status
git add README.md src
git commit -m "adiciona opcao de quantidade"
git push
git log --online
```

Erros frequentes e como agir

Problema	Como resolver
git não é reconhecido	Instale o Git for Windows, feche o PowerShell e abra uma nova janela.
A pasta já contém .git	Não execute git init novamente. Chame a professora para conferir se o projeto veio de outro repositório.
remote origin already exists	Não adicione outro origin sem verificar. Execute git remote -v e chame a professora se a URL estiver errada.
git push foi rejeitado	Não use --force. O repositório remoto pode conter um histórico diferente; pare e peça orientação.
Eclipse não encontra o projeto	Em Import > Existing Projects into Workspace, selecione a pasta que contém diretamente o arquivo .project.
Run As > Java Application não aparece	Confira se o projeto foi importado como projeto Java e se Principal.java contém o método main.
Há X vermelho no projeto	Abra Window > Show View > Problems e leia a mensagem antes de alterar o código.
README ou src não aparecem no GitHub	Execute git status, confira se os arquivos foram adicionados e faça commit + push novamente se necessário.

Checklist final de entrega

- ☐ Repositório individual da Aula 6 foi criado pelo Classroom 50.
- ☐ Código-base foi enviado ao repositório antes das alterações da prática.
- ☐ Projeto executa no Eclipse.
- ☐ Pacote controller existe e contém AlunoController.
- ☐ Principal cria e injeta o AlunoController.
- ☐ MenuAlunos usa AlunoController em vez de AlunoService diretamente.
- ☐ Opção 4 mostra a quantidade correta de alunos.
- ☐ README foi atualizado com responsabilidades e testes.
- ☐ Existem commits próprios do aluno no histórico.
- ☐ Últimas alterações foram enviadas com git push.